

Insert a Node at the Tail of a Linked List

Dynamic array solution C

hackerrank.com/challenges/insert-a-node-at-the-tail-of-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Linked Lists > Insert a Node at the Tail of a Linked List

Exit Full Screen View

Submissions

Leaderboard

Discussions

Editorial

STDIN

Function

5

size of linked list n = 5

141

linked list data values 141..474

302

164

530

474

Sample Output

141

302

164

530

474

Explanation

First the linked list is NULL. After inserting 141, the list is 141 -> NULL.

After inserting 302, the list is 141 -> 302 -> NULL.

After inserting 164, the list is 141 -> 302 -> 164 -> NULL.

After inserting 530, the list is 141 -> 302 -> 164 -> 530 -> NULL. After inserting 474, the list is 141 -> 302 -> 164 -> 530 -> 474 -> NULL, which is the final list.

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin)

Download

1 5

2 141

3 302

4 164

5 530

6 474

Your Output (stdout)

1 141

2 302

3 164

4 530

Sparse Arrays | HackerRank

Dynamic array solution C

hackerank.com/challenges/sparse-arrays/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Arrays > Sparse Arrays

Exit Full Screen View

Submissions

Leaderboard

Discussions

Editorial

abcedfscikljaskdjlneiascdn

Array queries

Sample Output 3

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Input (stdin)

Download

1	4
2	aba
3	baba
4	aba
5	xzxb
6	3
7	aba
8	xzxb
9	ab

Your Output (stdout)

1	2
---	---

Delete a Node | HackerRank

hackerank.com/challenges/delete-a-node-from-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Linked Lists > Delete a Node

Exit Full Screen View

Problem

Submissions

Leaderboard

ns

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.

Example

$l\text{list} = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$
 $position = 2$

After removing the node at position 2, $l\text{list}' = 0 \rightarrow 1 \rightarrow 3$.

Function Description

Complete the deleteNode function in the editor below.

deleteNode has the following parameters:

- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove

Returns

- SinglyLinkedListNode pointer: a reference to the head of the modified list

Input Format

The first line of input contains an integer n , the number of elements in the linked list.

Each of the next n lines contains an integer, the node data values in order.

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

4	2
5	19
6	7
7	4
8	15
9	9
10	3

Your Output (stdout)

1	20 6 2 7 4 15 9
---	-----------------

Expected Output

1	20 6 2 7 4 15 9
---	-----------------

Download

The screenshot shows a HackerRank challenge page for 'Array Manipulation' in full-screen mode. A large, semi-transparent congratulatory modal is centered on the screen, reading 'Congrats! You have earned your 2nd star.' and 'Continue'. The background shows the problem details, including constraints, sample input/output, and a code editor with a solution. The bottom of the screen shows a Windows taskbar with various icons and system information.

Browser Tabs: Array Manipulation | HackerRank, Dynamic array solution C

Address Bar: hackerrank.com/challenges/crush/problem?isFullScreen=true

Buttons: Set Google Chrome as your default browser and pin it to your taskbar, Set as default, Exit Full Screen View

HackerRank Header: Prepare > Data Structures > Arrays > Array Manipulation

Problem Constraints:

- $1 \leq a \leq b \leq n$
- $0 \leq k \leq 10^9$

Sample Input:

```
STDIN      Function
-----
5 3        arr[] size n = 5, queries[] size q = 3
1 2 100    queries = [[1, 2, 100], [2, 5, 100], [3,
2 5 100
3 4 100
```

Sample Output:

```
200
```

Explanation:

After the first update the list is 100 100 0 0 0.
 After the second update list is 100 200 100 100 100.
 After the third update list is 100 200 200 200 100.
 The maximum value is 200.

Code Editor:

```
28     free(arr);
29     return 0;
30 }
31
```

Problem Solving Modal:

Problem Solving
 ★★
 Congrats!
 You have earned your 2nd star.
 Continue

Background Progress:

0 points!
 You earned the 3rd star for your problem solving 45% 145/200

Footer:

27°C Mostly cloudy
 Search
 ENG IN
 18:28 21-11-2025

ms

The last line contains an integer, *position*, the position of the node to delete.

Print the Elements of a Linked L...Print linked list data

hackerrank.com/challenges/print-the-elements-of-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbarSet as default

HackerRank

Prepare > Data Structures > Linked Lists > Print the Elements of a Linked List

Exit Full Screen View

Submissions

Leaderboard

Discussions

Editorial

Constraints

Sample Input

Sample Output

Explanation

1 ≤ n ≤ 1000

1 ≤ list[i] ≤ 1000, where list[i] is the ith element of the linked list.

STDIN	Function
2	n = 2
16	first data value = 16
13	second data value = 13

16

13

There are two elements in the linked list. They are represented as 16 -> 13 -> NULL. So, the *printLinkedList* function should print 16 and 13 each on a new line.

Change ThemeLanguage C

```
1 > #include <assert.h>...
55 void printLinkedList(SinglyLinkedListNode* head) {
56     SinglyLinkedListNode* current = head;
57
58     // Traverse until the end of the list
59     while (current != NULL) {
60         printf("%d\n", current->data); // Print data
61         current = current->next;      // Move to next node
62     }
63 }
64
65 > int main() ...
```

Line: 64 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Insert a node at a specific position

hackerank.com/challenges/insert-a-node-at-a-specific-position-in-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

HackerRank

Prepare > Data Structures > Linked Lists > Insert a node at a specific position in a linked list

Exit Full Screen View

Problem

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head node of a linked list and an integer to insert at a certain position, create a new node with the given integer as its *data* attribute, insert this node at the desired position, and return the head node.

A position of 0 indicates the head, a position of 1 indicates one node away from the head, and so on. The head pointer given may be null, meaning that the initial list is empty.

Example

head refers to the first node in the list $1 \rightarrow 2 \rightarrow 3$

data = 4

position = 2

Insert a node at position 2 with *data* = 4. The new list is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Function Description

Complete the function *insertNodeAtPosition* with the following parameters:

- SinglyLinkedListNode pointer llist*: a reference to the head of the list
- data*: an integer value to insert as data in the new node
- position*: an integer position to insert the new node, zero-based indexing

Returns

- SinglyLinkedListNode pointer*: a reference to the head of the revised list

Submissions

Leaderboard

ns

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Sample Test case 2

Input (stdin)

Download

1	3
2	16
3	13
4	7
5	1
6	2

Your Output (stdout)

1	16 13 1 7
---	-----------

Expected Output

Download

1	16 13 1 7
---	-----------

Jim and the Skyscrapers | Hackerrank

Dynamic array solution C

hackerrank.com/challenges/jim-and-the-skyscrapers/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Advanced > Jim and the Skyscrapers

Exit Full Screen View

Submit

6
3 2 1 2 3 3

Sample Output #00

8

Sample Input #01

3
1 1000 1

Sample Output #01

0

Explanation

First testcase: (1, 5), (1, 6) (5, 6) and (2, 4) and the routes in the opposite directions are the only valid routes.

Second testcase: (1, 3) and (3, 1) could have been valid, if there wasn't a big skyscraper with height 1000 between them.

Leaderboard

Discussions

Editorial

Change Theme

Language C

```
5  int main() {  
22  while (top >= 0 && sh[top] < h[i]) {  
23      top--;  
24  }  
25  
26  if (top >= 0 && sh[top] == h[i]) {  
27      // Same height -> valid pairs formed  
28      ans += freq[top];  
29      freq[top]++; // Increase frequency  
30  } else {  
31      // New height entry  
32      top++;  
33      sh[top] = h[i];  
34      freq[top] = 1;  
35  }  
36  }  
37  
38  printf("%lld\n", ans);  
39  return 0;  
40 }  
41
```

Line: 41 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Insert a Node at the Tail of a Li...Dynamic array solution C

hackerrank.com/challenges/insert-a-node-at-the-tail-of-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbarSet as default

HackerRank

Prepare > Data Structures > Linked Lists > Insert a Node at the Tail of a Linked List

Exit Full Screen View

Subm

Leaderboard

Discussions

Editorial

STDIN	Function
5	size of linked list n = 5
141	linked list data values 141..474
302	
164	
530	
474	

Sample Output

141
302
164
530
474

Explanation

First the linked list is NULL. After inserting 141, the list is 141 -> NULL.

After inserting 302, the list is 141 -> 302 -> NULL.

After inserting 164, the list is 141 -> 302 -> 164 -> NULL.

After inserting 530, the list is 141 -> 302 -> 164 -> 530 -> NULL. After inserting 474, the list is 141 -> 302 -> 164 -> 530 -> 474 -> NULL, which is the final list.

Change ThemeLanguageC

```
54 SinglyLinkedListNode* insertNodeAtTail(SinglyLinkedListNode* head, int data) {
55     SinglyLinkedListNode* newNode = create_singly_linked_list_node(data);
56
57     // If list is empty, new node becomes head
58     if (head == NULL) {
59         return newNode;
60     }
61
62     // Otherwise, traverse to the end of the list
63     SinglyLinkedListNode* temp = head;
64     while (temp->next != NULL) {
65         temp = temp->next;
66     }
67
68     // Insert new node at the end
69     temp->next = newNode;
70
71     return head;
72 }
73
74
75
```

Line: 74 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Print in Reverse | HackerRank

hackerank.com/challenges/print-the-elements-of-a-linked-list-in-reverse/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Linked Lists > Print in Reverse

Exit Full Screen View

Problem

Submissions

Leaderboard

ns

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head of a singly-linked list, print each *data* value from the reversed list. If the given list is empty, do not print anything.

Example

*head** refers to the linked list with *data* values $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{NULL}$

Print the following:

3
2
1

Function Description

Complete the `reversePrint` function in the editor below.

`reversePrint` has the following parameters:

- `SinglyLinkedListNode` pointer `head`: a reference to the head of the list

Prints

The *data* values of each node in the reversed list.

Input Format

The first line of input contains *t*, the number of test cases.

The input of each test case is as follows:

- The first line contains an integer, the number of elements in the list.

Change Theme

Language

c

🕒

🔍

⋮

```
1 > #include <assert.h>...
67
68 void reversePrint(SinglyLinkedListNode* head) {
69     if (head == NULL) return; // base case: empty list
70     reversePrint(head->next); // recursively print the rest of the
71     list
72     printf("%d\n", head->data); // print current node after recursion
73 }
74
75 > int main()...
```

Line: 67 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Insert a node at the head of a linked list

hackerrank.com/challenges/insert-a-node-at-the-head-of-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Linked Lists > Insert a node at the head of a linked list

Exit Full Screen View

Submit

975
392
484
383

Explanation

Initially the list is NULL. After inserting 383, the list is 383 -> NULL.
After inserting 484, the list is 484 -> 383 -> NULL.
After inserting 392, the list is 392 -> 484 -> 383 -> NULL.
After inserting 975, the list is 975 -> 392 -> 484 -> 383 -> NULL.
After inserting 321, the list is 321 -> 975 -> 392 -> 484 -> 383 -> NULL.

Leaderboard

Discussions

Editorial

Change Theme Language C

```
1 > #include <assert.h>...
55 SinglyLinkedListNode* insertNodeAtHead(SinglyLinkedListNode* llist, int
    data) {
56     // Create a new node
57     SinglyLinkedListNode* newNode = malloc(sizeof(SinglyLinkedListNode));
58     newNode->data = data;
59
60     // Make the new node point to current head
61     newNode->next = llist;
62
63     // Return the new head
64     return newNode;
65 }
66
67
68 > int main()|...
```

Line: 68 Col: 11

Upload Code as File

Test against custom input

Run Code

Submit Code

Insert a node at a specific position

hackerank.com/challenges/insert-a-node-at-a-specific-position-in-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

HackerRank

Prepare > Data Structures > Linked Lists > Insert a node at a specific position in a linked list

Exit Full Screen View

Problem

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head node of a linked list and an integer to insert at a certain position, create a new node with the given integer as its *data* attribute, insert this node at the desired position, and return the head node.

A position of 0 indicates the head, a position of 1 indicates one node away from the head, and so on. The head pointer given may be null, meaning that the initial list is empty.

Example

head refers to the first node in the list $1 \rightarrow 2 \rightarrow 3$

data = 4

position = 2

Insert a node at position 2 with *data* = 4. The new list is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Function Description

Complete the function *insertNodeAtPosition* with the following parameters:

- SinglyLinkedListNode* pointer *llist*: a reference to the head of the list
- data*: an integer value to insert as data in the new node
- position*: an integer position to insert the new node, zero-based indexing

Returns

- SinglyLinkedListNode* pointer: a reference to the head of the revised list

Change Theme

Language

C

67

SinglyLinkedListNode* insertNodeAtPosition(
82
83 // Case 2: Insert at given position (middle or end)
84 SinglyLinkedListNode* current = head;
85 int index = 0;
86
87 // Traverse to the node before the insertion point
88 while (index < position - 1 && current != NULL) {
89 current = current->next;
90 index++;
91 }
92
93 // Now 'current' points to (position-1)th node
94 newNode->next = current->next;
95 current->next = newNode;
96
97 // Return unchanged head
98 return head;
99 }
100
101

Line: 100 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Delete a Node | HackerRank

hackerank.com/challenges/delete-a-node-from-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar

Set as default

HackerRank

Prepare > Data Structures > Linked Lists > Delete a Node

Exit Full Screen View

Problem

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.

Example

$l\text{list} = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

$\text{position} = 2$

After removing the node at position 2, $l\text{list}' = 0 \rightarrow 1 \rightarrow 3$.

Function Description

Complete the deleteNode function in the editor below.

deleteNode has the following parameters:

- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove

Returns

- SinglyLinkedListNode pointer: a reference to the head of the modified list

Input Format

The first line of input contains an integer n , the number of elements in the linked list.

Each of the next n lines contains an integer, the node data values in order.

Change Theme Language c

```
1 > #include <assert.h>...
67 SinglyLinkedListNode* deleteNode(SinglyLinkedListNode* llist, int
   position) {
68
69     // Case 1: Delete head node (position = 0)
70     if (position == 0) {
71         SinglyLinkedListNode* temp = llist; // current head
72         llist = llist->next;                // move head forward
73         free(temp);                         // free old head
74         return llist;                      // return new head
75     }
76
77     // Case 2: Delete node in middle or end
78     SinglyLinkedListNode* current = llist;
79     int index = 0;
80
81     // Traverse to one node before the node to delete
82     while (index < position - 1 && current != NULL) {
83         current = current->next;
84         index++;
85     }
```

Line: 67 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

The image shows a web browser window displaying a HackerRank challenge titled "Crush". The problem involves an array of 100 zeros and three queries that update specific elements. The constraints are $1 \leq a \leq b \leq n$ and $0 \leq k \leq 10^9$. The sample input shows three queries: [1, 2, 100], [2, 5, 100], and [3, 4, 100]. The sample output is 200. The explanation states that after the first update, the list is 100 100 0 0 0. After the second update, it is 100 200 100 100 100. After the third update, it is 100 200 200 200 100. The maximum value is 200.

The code is written in C++ and uses a prefix sum array to efficiently calculate the maximum value. It starts with an array of 100 zeros. For each query, it adds the value k to the element at index a and subtracts k from the element at index b+1. Finally, it iterates through the array to find the maximum value.

```
4 int main() {
12     scanf("%lld %lld %lld", &a, &b, &k);
13
14     arr[a] += k; // Add k at start
15     arr[b + 1] -= k; // Subtract k after end
16 }
17
18 long long max = 0, current = 0;
19
20 for (long long i = 1; i <= n; i++) {
21     current += arr[i]; // Prefix sum reconstructs array
22     if (current > max)
23         max = current;
24 }
25
26 printf("%lld\n", max);
27
28 free(arr);
29 return 0;
30 }
31 }
```